

Workshop 7: OpenTelemetry — auto-instrumentacja, śledzenie i korelacja sygnałów

Wymagania wstępne

- Zakończony Workshop 6: Prometheus, Grafana, Loki, Promtail i Tempo działające w namespace `monitoring` ; Online Boutique wdrożone w namespace `boutique`
 - Zmienna `NODE_IP` ustawiona w terminalu (patrz sekcja „Przed rozpoczęciem” w Workshop 6)
-

Przed rozpoczęciem

Sprawdź, czy stos monitoringu nadal działa:

```
kubectl get pods -n monitoring
kubectl get pods -n boutique
```

Wszystkie pody powinny być w stanie `Running` . Jeśli klaster był zatrzymany i uruchomiony ponownie, poczekaj 2-3 minuty, aż wszystko wróci do działania.

Przywróć zmienną `NODE_IP` , jeśli sesja terminala jest nowa.

Jeśli klaster został zbudowany przy użyciu skryptów GCP:

```
export NODE_IP=$(cd ~/prin-2026-observability/scripts/gcp-scripts/terraform &&
terraform output -json worker_public_ips | python3 -c "import sys,json;
print(json.load(sys.stdin)[0])")
echo $NODE_IP
```

Jeśli klaster został skonfigurowany inną metodą, ustaw adres IP ręcznie:

```
export NODE_IP=<zewnętrzny-ip-węzła-roboczego>
echo $NODE_IP
```

Czym jest OpenTelemetry?

Workshop 6 obejmował metryki i logi. Dziś dodajesz trzeci filar — **śledzenie rozproszone** — a następnie łączysz wszystkie trzy sygnały, by móc nawigować między nimi z poziomu jednego zdarzenia.

Dlaczego OpenTelemetry? Przed powstaniem OTel każdy dostawca narzędzi obserwowalności miał własny SDK. Instrumentacja pod Datadog oznaczała vendor lock-in na poziomie kodu. OpenTelemetry (OTel) to projekt CNCF ze statusem graduated, który definiuje jeden standard niezależny od dostawcy:

- **SDK / API** — instalowany w aplikacji (lub do niej wstrzykiwany); produkuje spany, metryki i rekordy logów
- **OTLP** — protokół przesyłu danych (gRPC na porcie 4317, HTTP na porcie 4318)
- **Collector** — samodzielna usługa odbierająca, przekształcająca i przekierowująca telemetry do dowolnego backendu

Efekt: instrumentujesz raz, wysyłasz wszędzie. Backend (Tempo, Jaeger, Datadog) staje się wyborem konfiguracyjnym, a nie zmianą kodu.

Kontekst telco: W środowiskach 5G Core każda Network Function (AMF, SMF, UPF) może emitować ślady OTel dla każdej sesji subskrybenta. Pozwala to operatorom prześledzić procedurę dołączania pojedynczego UE przez AMF → SMF → UPF — dokładnie ta sama technika, którą zastosujesz dziś, tylko z inaczej nazwanymi usługami.

Co zobaczysz podczas tych zajęć

Zinstrumentujesz cztery usługi Online Boutique w dwóch językach:

Usługa	Język	Auto-instrumentacja
paymentservice	Node.js	inject-nodejs
currencyservice	Node.js	inject-nodejs
emailservice	Python	inject-python
recommendationservice	Python	inject-python

Zajęcia składają się z ośmiu ćwiczeń:

1. Wdrożenie OTel Operator
2. Wdrożenie OTel Collector — pipeline wielosygnałowy (ślady, logi, metryki)

3. Włączenie auto-instrumentacji na czterech usługach w dwóch językach
 4. Zapytania o ślady za pomocą **TraceQL** — w tym łańcuchy śladów między usługami
 5. Uruchomienie **service graph** i budowa dashboardu RED ze span metrics
 6. Konfiguracja **korelacji sygnałów** — przechodzenie ze śladu do logów i metryk
 7. Konfiguracja **Alertmanager** — alerty na podstawie metryk opóźnień ze spanów
 8. **Challenge** — wzbogacenie pipeline OTel o własne procesory
-

Ćwiczenie 1 — Wdrożenie OpenTelemetry Operator

Kubernetes Operator to wzorzec pakowania wiedzy operacyjnej w kod. Operator obserwuje Custom Resource Definitions (CRDs) i reaguje na nie — automatycznie tworząc Deployments, Serwisy i inne zasoby. Zarządzasz operatorem przez YAML; on zajmuje się szczegółami implementacji.

OTel Operator zarządza dwoma CRDs:

- **OpenTelemetryCollector** — definiuje instancję Collectora; operator tworzy Deployment i Service
- **Instrumentation** — definiuje, które agenty SDK wstrzyknąć i gdzie wysyłać dane; admission webhook operatora wstrzykuje je do podów automatycznie

1.1 Dodaj repozytorium Helm

```
helm repo add open-telemetry https://open-telemetry.github.io/opentelemetry-helm-charts
helm repo update
```

1.2 Zainstaluj operator

```
helm install opentelemetry-operator open-telemetry/opentelemetry-operator \
  --namespace otel-system \
  --create-namespace \
  --set manager.collectorImage.repository=otel/opentelemetry-collector-contrib \
  --set admissionWebhooks.certManager.enabled=false \
  --set admissionWebhooks.autoGenerateCert.enabled=true \
  --wait --timeout 5m
```

`collectorImage.repository=otel/opentelemetry-collector-contrib` — dystrybucja `contrib` zawiera eksportery dla Tempo, Loki i Prometheusa, a także zaawansowane procesory używane w Ćwiczeniu 8.

`certManager.enabled=false` + `autoGenerateCert.enabled=true` — admission webhook operatora wymaga TLS. Operator generuje własny certyfikat, eliminując zależność od cert-manager.

1.3 Weryfikacja

```
kubectl get pods -n otel-system
kubectl get crd | grep opentelemetry
```

Oczekiwany wynik: pod `opentelemetry-operator-*` w stanie `Running` oraz zarejestrowane CRD `opentelemetrycollectors.opentelemetry.io` i `instrumentations.opentelemetry.io`.

Ćwiczenie 2 — Wdrożenie OTEL Collector

OTel Collector to centralny punkt routingu całej telemetry. Działa zgodnie z modelem **pipeline**:

```
Receivers → Processors → Exporters
```

W tym ćwiczeniu wdrażasz Collector wstępnie skonfigurowany z trzema pipeline'ami — po jednym na sygnał: ślady do Tempo, logi do Loki, metryki do Prometheusa.

Uwaga: W tych zajęciach dane przesyła tylko pipeline ze śladami. Pipeline dla logów i metryk są podłączone, ale nieaktywne — usługi Online Boutique nie są kompatybilne z OTEL log bridge, a używane wersje SDK domyślnie nie emitują metryk runtime. W środowisku produkcyjnym z natywnym logowaniem OTEL wszystkie trzy pipeline'y byłyby aktywne i mogłyby zastąpić Promtail oraz część scrapingu Prometheusa skonfigurowanego w Workshop 6.

2.1 Zastosuj manifest Collectora

```
kubectl apply -f ~/prin-2026-observability/manifests/otel-collector.yaml
```

Manifest łączy trzy pipeline'y ze stosem monitoringu działającym od Workshop 6:

```

exporters:
  otlp_grpc/tempo:
    endpoint: tempo.monitoring.svc:4317      # ślady → Tempo
  otlp_http/loki:
    endpoint: http://loki.monitoring.svc:3100/otlp # logi → Loki
  prometheusremotewrite:
    endpoint: http://kube-prometheus-stack-prometheus.monitoring.svc:9090/api/v1/
write

service:
  pipelines:
    traces: { receivers: [otlp], processors: [memory_limiter, batch], exporters:
[otlp_grpc/tempo, debug] }
    logs:   { receivers: [otlp], processors: [batch], exporters: [otlp_http/
loki] }
    metrics: { receivers: [otlp], processors: [batch], exporters: [prometheusremote
write] }

```

2.2 Weryfikacja

```

kubectl get pods -n otel-system
kubectl get svc -n otel-system
kubectl logs -n otel-system -l app.kubernetes.io/component=opentelemetry-collector | head -20

```

Oczekiwany wynik:

- Pod `otel-collector-*` — `Running`
- Serwis `otel-collector` — porty `4317` (gRPC) i `4318` (HTTP)
- W logach widoczny komunikat: `Everything is ready. Begin running and processing data.`

2.3 Potwierdź, że Tempo jest puste przed instrumentacją

Otwórz **Grafana** → **Explore** → **Tempo** → **Search** → **Run query** (zakres czasu: Last 15 minutes).

Ślady nie powinny się jeszcze pojawić. To jest punkt odniesienia — po Ćwiczeniu 3 to samo zapytanie zwróci spany dla każdego żądania obsługowanego przez zinstrumentowane usługi.

Ćwiczenie 3 — Włączenie auto-instrumentacji

Auto-instrumentacja wstrzykuje agenta SDK OTel do podów aplikacji bez modyfikacji kodu źródłowego. **Admission webhook** operatora przechwytuje tworzenie poda i wstrzykuje:

1. **Init container** kopiujący plik binarny agenta do współdzielonego wolumenu
2. **Zmienne środowiskowe** nakazujące środowisku uruchomieniowemu załadowanie agenta

Mechanizm wstrzykiwania różni się w zależności od języka:

Język	Sposób ładowania agenta
Node.js	<code>NODE_OPTIONS=-require /otel-auto-instrumentation-nodejs/autoinstrumentation.js</code>
Python	<code>PYTHONSTARTUP=/otel-auto-instrumentation-python/sitecustomize.py</code>
Java	<code>JAVA_TOOL_OPTIONS=-javaagent:/otel-auto-instrumentation-java/javaagent.jar</code>

3.1 Zastosuj zasób Instrumentation

```
kubectl apply -f ~/prin-2026-observability/manifests/otel-instrumentation.yaml
kubectl get instrumentation -n boutique
```

Zasób konfiguruje agenta SDK — wskazuje endpoint HTTP Collectora i ustawia propagację W3C:

```
spec:
  exporter:
    endpoint: http://otel-collector.otel-system.svc:4318
  propagators:
    - tracecontext
    - baggage
  sampler:
    type: parentbased_traceidratio
    argument: "1.0"
```

`parentbased_traceidratio` z wartością `"1.0"` — próbkowanie 100% śladów, odpowiednie dla środowiska laboratoryjnego.

3.2 Dodaj adnotacje do czterech Deploymentów

Usługi Node.js:

```
kubectl patch deployment paymentservice -n boutique -p \
  '{"spec":{"template":{"metadata":{"annotations":{"instrumentation.opentelemetry.io/inject-nodejs":"true"}}}}}'

kubectl patch deployment currencyservice -n boutique -p \
  '{"spec":{"template":{"metadata":{"annotations":{"instrumentation.opentelemetry.io/inject-nodejs":"true"}}}}}'
```

Usługi Python:

```
kubectl patch deployment emailservice -n boutique -p \
  '{"spec":{"template":{"metadata":{"annotations":{"instrumentation.opentelemetry.io/inject-python":"true"}}}}}'

kubectl patch deployment recommendationservice -n boutique -p \
  '{"spec":{"template":{"metadata":{"annotations":{"instrumentation.opentelemetry.io/inject-python":"true"}}}}}'
```

Każda adnotacja wyzwala rolling restart. Admission webhook uruchamia się przy każdym tworzeniu nowego poda i wstrzykuje odpowiedniego agenta.

3.3 Popraw liveness probe dla usług Python

Agent OTel dla Pythona wprowadza narzut podczas uruchamiania — hook `sitecustomize.py` i modyfikacje biblioteki gRPC muszą się zakończyć, zanim serwer zacznie przyjmować połączenia. Oryginalne Deploymenty mają `initialDelaySeconds: 0`, co oznacza, że liveness probe uruchamia się natychmiast i zatrzymuje kontener, zanim Python zdąży się załadować przy ograniczonych zasobach CPU.

Popraw obie usługi Python:

```
kubectl patch deployment emailservice -n boutique -p \
  '{"spec":{"template":{"spec":{"containers":[{"name":"server","livenessProbe":{"initialDelaySeconds":30},"readinessProbe":{"initialDelaySeconds":30}}]}}}}'

kubectl patch deployment recommendationservice -n boutique -p \
  '{"spec":{"template":{"spec":{"containers":[{"name":"server","livenessProbe":{"initialDelaySeconds":30},"readinessProbe":{"initialDelaySeconds":30}}]}}}}'
```

Usługi Node.js (`paymentservice` , `currencyservice`) nie wymagają tej poprawki — agent OTel dla Node.js ładuje się szybciej i obecne opóźnienie liveness probe jest wystarczające.

3.4 Weryfikacja wstrzyknięcia

Poczekaj, aż wszystkie cztery pody wrócą do działania:

```
kubectl get pods -n boutique \
  -l 'app in (paymentservice,currencyservice,emailservice,recommendationservice)' -w
```

Gdy wszystkie są w stanie `Running` (Ctrl+C), porównaj sposób instrumentacji w obu językach:

```
# Node.js – agent ładowany przez NODE_OPTIONS
kubectl describe pod -n boutique -l app=paymentservice | grep -E "Init Containers|
NODE_OPTIONS" -A5

# Python – agent ładowany przez PYTHONSTARTUP
kubectl describe pod -n boutique -l app=emailservice | grep -E "Init Containers|
PYTHONSTARTUP" -A5
```

Oba pody mają init container `opentelemetry-auto-instrumentation-*`, ale zmienna środowiskowa uruchamiająca agenta różni się w zależności od języka. W obu przypadkach agent jest ładowany przed uruchomieniem kodu aplikacji.

3.5 Sprawdź, czy ślady docierają

Wygeneruj ruch — przeglądaj sklep i przejdź przez proces zakupu:

```
http://<NODE_IP>:30080
```

Sprawdź, czy spany docierają do Collectora:

```
kubectl logs -n otel-system -l app.kubernetes.io/component=opentelemetry-collector --
tail=20
```

Przejdź do **Grafana** → **Explore** → **Tempo** → **Search** → **Run query** — wszystkie cztery usługi powinny teraz pojawiać się na liście.

Ćwiczenie 4 — Zapytania o ślady z TraceQL

Tempo ma własny język zapytań — **TraceQL** — który nawiązuje do podejścia, które już znasz z PromQL i LogQL: selektor `{...}` filtrujący zwracane spany.

Otwórz **Grafana** → **Explore** → **Tempo** i przełącz się z zakładki **Search** na zakładkę **TraceQL**.

4.1 Podstawowe selektory

Wybierz wszystkie spany z konkretnej usługi:

```
{resource.service.name="paymentservice"}
```

Filtruj do powolnych spanów:

```
{resource.service.name="paymentservice" && duration > 2ms}
```

Filtruj do spanów z błędami:

```
{status=error}
```

Brak wyników dla filtrów według czasu trwania i błędów jest normalny przy niskim ruchu. Przeglądaj sklep i przejdź przez zakup kilka razy, aby wygenerować więcej spanów.

4.2 Zapytania wielousługowe

Wybierz spany ze wszystkich czterech zinstrumentowanych usług za pomocą wyrażenia regularnego:

```
{resource.service.name=~"paymentservice|currencyservice|emailservice|recommendationservice"}
```

Posortuj wyniki według **Duration** (kliknij nagłówek kolumny). Zobaczysz spany ze wszystkich czterech usług wymieszane razem. Kliknij dowolny wynik — otwórz widok waterfall.

4.3 Porównaj atrybuty między językami

Kliknij span z usługi `paymentservice` (Node.js) i przejrzyj panel **Attributes**. Zanotuj, jakie atrybuty są obecne.

Następnie znajdź span z usługi `emailservice` (Python) i porównaj zestawy atrybutów. Obie usługi powinny mieć wspólne:

- `k8s.pod.name`, `k8s.namespace.name` — wstrzykiwane przez operator jako zmienne środowiskowe
- `rpc.method`, `rpc.grpc.status_code` — z biblioteki instrumentacji gRPC

Różnice pojawiają się w atrybutach specyficznych dla danego SDK (wersja środowiska uruchomieniowego, wersja biblioteki instrumentacji). To pokazuje, dlaczego

`resource.service.name` jest stabilnym kluczem między usługami w TraceQL — jest ustawiany spójnie niezależnie od języka.

4.4 Filtruj według rodzaju spana

`kind` to wbudowany atrybut TraceQL określający, jak span został wygenerowany:

- `server` — usługa otrzymała przychodzące żądanie
- `client` — usługa wykonała wychodzące wywołanie

Znajdź tylko spany po stronie serwera we wszystkich czterech usługach:

```
{resource.service.name=~"paymentservice|currencyservice|emailservice|recommendationservice" && kind=server}
```

To są punkty wejścia — jeden na każde przychodzące żądanie obsłużone przez daną usługę. Kliknij wynik i przejrzyj, jakie informacje o nadrzędnym trace (jeśli istnieją) są widoczne w widoku waterfall.

Ćwiczenie 5 — Service graph i dashboard RED

Service graph to mapa topologii generowana z danych śladów. **Dashboard RED** (Rate, Errors, Duration) to standardowy widok SRE dla kondycji usługi. Oba wywodzą się z tego samego źródła: **generatora metryk** Tempo, który odczytuje przychodzące spany i generuje metryki Prometheusa — jeden licznik i jeden histogram na usługę.

5.1 Włącz remote write w Prometheusie

```
helm upgrade kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --reuse-values \
  --set prometheus.prometheusSpec.enableRemoteWriteReceiver=true
```

Poczekaj na restart Prometheusa:

```
kubectl rollout status statefulset/prometheus-kube-prometheus-stack-prometheus -n monitoring
```

5.2 Włącz generator metryk Tempo

```
cat ~/prin-2026-observability/manifests/tempo-metrics.yaml

helm upgrade tempo grafana/tempo \
  --namespace monitoring \
  --reuse-values \
  --values ~/prin-2026-observability/manifests/tempo-metrics.yaml
```

Poczekaj na restart Tempo:

```
kubectl rollout status statefulset/tempo -n monitoring
```

5.3 Podłącz service graph w Grafanie

Przejdź do **Connections** → **Data sources** → **Tempo** → **edit** → **Service graph**.

- **Data source:** Prometheus

Kliknij **Save & test**. Przejdź do **Explore** → **Tempo** → **Service Graph** → **Run query**.

Cztery zinstrumentowane usługi pojawiają się jako węzły, wraz z węzłem `user` reprezentującym niezinstrumentowanych wywołujących — `frontend` i `checkoutservice` nie mają adnotacji, więc pojawiają się jako jedno nienazwane źródło. Kliknij krawędź, aby zobaczyć szybkość wywołań i histogram opóźnień dla danej pary usług.

Kompletność service graph zależy od zakresu Twojej instrumentacji. W Ćwiczeniu 8 zinstrumentujesz piątą usługę — obserwuj, jak zmieni się graf.

5.4 Zbuduj dashboard RED ze span metrics

Generator metryk Tempo zapisuje dane do Prometheusa przez remote write:

- `traces_spanmetrics_calls_total` — licznik, etykiety: `service`, `span_name`, `status_code`
- `traces_spanmetrics_latency_bucket` — histogram z tymi samymi etykietami

W Grafanie utwórz nowy dashboard i dodaj trzy panele.

Panel 1 — Liczba żądań (Time series)

```
sum(rate(traces_spanmetrics_calls_total[1m])) by (service)
```

Ustaw legendę na `{{service}}`.

Panel 2 — Współczynnik błędów (Time series)

```
sum(rate(traces_spanmetrics_calls_total{status_code="STATUS_CODE_ERROR"}[1m])) by
(service)
/
sum(rate(traces_spanmetrics_calls_total[1m])) by (service)
```

Ustaw jednostkę na **Percent (0-1)**. Zdrowa usługa pokazuje wartość bliską zeru.

Panel 3 — Opóźnienie P99 (Time series)

```
histogram_quantile(0.99,
  sum(rate(traces_spanmetrics_latency_bucket[1m])) by (le, service)
)
```

Ustaw jednostkę na **seconds**.

Zapisz dashboard jako **OTel RED Dashboard**. Zostaw go otwarty — będziesz obserwować zmiany panelu P99 w Ćwiczeniu 7.

5.5 Wygeneruj błędy, aby przetestować panel współczynnika błędów

Panel współczynnika błędów pokazuje spany `STATUS_CODE_ERROR` z czterech zinstrumentowanych usług. Aby wywołać błędy, przeskaluj do zera usługę, od której zależy `recommendationservice`:

```
kubectl scale deployment productcatalogservice -n boutique --replicas=0
```

Przeglądaj sklep — sekcja rekomendacji przestanie działać. Obserwuj panel współczynnika błędów: błędy `recommendationservice` pojawiają się w ciągu minuty, gdy jej wychodzące wywołania gRPC do `productcatalogservice` zaczynają zwracać błędy.

Przywróć po zakończeniu:

```
kubectl scale deployment productcatalogservice -n boutique --replicas=1
```

Martwe pole instrumentacji: Przeskaluj zamiast tego `redis-cart` do zera — aplikacja stanie się w dużej mierze nieużywalna (koszyk, kasa i większość przepływów użytkownika przestają działać), ale dashboard RED nie pokaże żadnych błędów. `cartservice` nie jest zinstrumentowany, więc jego awarie nie produkują spanów i są dla dashboardu RED całkowicie niewidoczne. System jest uszkodzony, a dashboard pozostaje zielony. Przywróć: `kubectl scale deployment redis-cart -n boutique --replicas=1`

Ćwiczenie 6 — Korelacja sygnałów

Do tej pory każdy sygnał funkcjonuje w swoim własnym widoku. W tym ćwiczeniu skonfigurujesz Grafanę tak, aby je łączyła — ze śladu możesz przejść bezpośrednio do logów i metryk poda, który go wyprodukował.

6.1 Ze śladu do logów

Przejdź do **Connections** → **Data sources** → **Tempo** → **edit** → **Trace to logs**.

- **Data source:** Loki
- **Tags:** klucz `k8s.pod.name`, wartość `pod`
- **Span start time shift:** `-1m`
- **Span end time shift:** `1m`

Kliknij **Save & test**.

Test: przejdź do **Explore** → **Tempo** → **Search**, znajdź ślad usługi `paymentservice`, otwórz go, kliknij dowolny span. Obok spana pojawi się przycisk **Logs** — kliknij go. Grafana otwiera Loki z filtrem ograniczonym do logów tego poda w dokładnym przedziale czasowym spana.

Grafana odczytuje `k8s.pod.name` z atrybutów spana, mapuje go na etykietę `pod` w Loki i generuje zapytanie `{pod="paymentservice-xxxxx"}` z zakresem czasowym spana. Identyfikator trace w logach nie jest potrzebny — korelacja opiera się na metadanych.

6.2 Ze śladu do metryk

W ustawieniach źródła danych Tempo przewiń do sekcji **Trace to metrics**.

- **Data source:** Prometheus
- Dodaj zapytanie — nazwa `CPU usage` :

```
sum(rate(container_cpu_usage_seconds_total{pod='${__span.tags["k8s.pod.name"]}'}[1m]))
by (pod)
```

- Dodaj zapytanie — nazwa `Pod restarts` :

```
round(increase(kube_pod_container_status_restarts_total{pod='${__span.tags["k8s.pod.name"]}'}[10m]))
```

Kliknij **Save & test**.

Test: otwórz span usługi `paymentservice` — obok przycisku **Logs** pojawi się teraz także przycisk **Metrics**. Kliknięcie go otwiera Explore Prometheusa z zapytaniem CPU wypełnionym dla tego poda.

Ćwiczenie 7 — Alertmanager

Metryki opóźnień pochodzące z danych spanów docierają już do Prometheusa. W tym ćwiczeniu skonfigurujesz alert uruchamiający się przy wysokim opóźnieniu P99 i zweryfikujesz cały łańcuch: PrometheusRule → routing Alertmanager.

7.1 Zastosuj PrometheusRule

Reguła uruchamia alert, gdy opóźnienie P99 spanów usługi `recommendationservice` przekroczy 50 ms:

```
kubectl apply -f ~/prin-2026-observability/manifests/boutique-alert.yaml
```

Ustaw port-forward do Prometheusa i sprawdź, czy reguła jest załadowana:

```
kubectl port-forward -n monitoring svc/kube-prometheus-stack-prometheus 9090:9090 &
sleep 2 && curl -s localhost:9090/api/v1/rules | python3 -m json.tool | grep -E '"HighSpanLatency"' -A5 -B5
```

Oczekiwany wynik:

```
"name": "span-metrics",
"rules": [
  {
    "state": "inactive",
    "name": "HighSpanLatency",
    "query": "histogram_quantile(0.99, ...) > 0.05",
    "duration": 30,
```

`state: inactive` potwierdza, że reguła jest załadowana, a warunek jest obecnie fałszywy — to prawidłowy stan w tym momencie.

7.2 Zastosuj AlertmanagerConfig

```
kubectl apply -f ~/prin-2026-observability/manifests/boutique-alertmanager.yaml
```

Sprawdź, czy Alertmanager odebrał konfigurację routingu:

```
kubectl port-forward -n monitoring svc/kube-prometheus-stack-alertmanager 9093:9093 &
sleep 2 && curl -s localhost:9093/api/v2/status | python3 -m json.tool | grep "boutique"
```

7.3 Wyzwól alert za pomocą StressChaos

Obciążenie CPU nasycuje przetwarzanie usługi `recommendationservice` — jej handlery gRPC działają wolniej, co bezpośrednio zwiększa czas trwania spanów po stronie serwera rejestrowanych przez agenta OTel.

```
kubectl apply -f ~/prin-2026-observability/manifests/recommendation-latency.yaml
```

Obserwuj z dwóch widoków:

1. **Dashboard RED** (Ćwiczenie 5) — opóźnienie P99 dla `recommendationservice` przekracza 50 ms w ciągu 1-2 minut
2. **Alerty Prometheusa** — odpytuj do czasu uruchomienia reguły:

```
watch -n 10 'curl -s localhost:9090/api/v1/alerts | python3 -m json.tool | grep -E
"\alertname\|state\|"'
```

Wynik jest pusty, dopóki reguła nie opuści stanu `inactive` — to zachowanie jest oczekiwane. Reguła przechodzi `inactive` → `pending` → `firing` (łącznie ok. 60–90 s)

Gdy `HighSpanLatency` pokazuje `"state": "firing"`, cały łańcuch działa:

```
opóźnienie spana (Tempo) → span metrics (Prometheus) → PrometheusRule → Alertmanager
```

Posprzątaj po zakończeniu:

```
kubectl delete -f ~/prin-2026-observability/manifests/recommendation-latency.yaml
# Zatrzymaj port-forwardy uruchomione w 7.1 i 7.2
kill %1 %2 2>/dev/null; true
```

Ćwiczenie 8 — Challenge: Wzbogacenie pipeline OTel

To ćwiczenie nie zawiera instrukcji krok po kroku. Skorzystaj z tego, czego się nauczyłeś.

Część A — Instrumentacja `adservice`

`adservice` to usługa Java w Online Boutique. Obecnie nie jest zinstrumentowana — żadne spany nie pojawiają się dla niej w Tempo, a ona sama jest nieobecna w service graph.

Twoim celem: spraw, żeby się pojawiła. Znasz wzorzec z Ćwiczenia 3. Klucz adnotacji dla Javy jest inny — ustal, który to, i zastosuj go do Deploymentu `adservice`.

Po restarcie sprawdź:

- Spany z `adservice` pojawiają się w Tempo (`{resource.service.name="adservice"}`)
- Service graph pokazuje `adservice` jako węzeł (może minąć chwila przy małym ruchu)
- Przejrzyj wynik `kubectl describe pod` — porównaj, jak wstrzykiwanie agenta Java różni się od Node.js i Pythona

Część B — Dodaj procesory metadanych do Collectora

Obecnie żaden span w Tempo nie zawiera informacji o tym, z którego klastra pochodzi ani która grupa studentów go wygenerowała. W środowisku współdzielonym lub wieloklastrowym uniemożliwia to filtrowanie według pochodzenia.

Twoim celem: zmodyfikuj zasób `OpenTelemetryCollector` tak, aby każdy span wygenerowany przez dowolną zinstrumentowaną usługę automatycznie zawierał dodatkowe atrybuty:

1. **Metadane chmury i hosta** — wykrywane automatycznie ze środowiska GCP (region, strefa dostępności, nazwa hosta), bez hardkodowania
2. `student.group` — ustawione na nazwę grupy złożoną z Waszych nazwisk (np. `"group-KowalskiNowak"`)
3. `workshop` — ustawione na `"prin-2026-w7"`

Wskazówki:

- Dystrybucja OTEL Collector contrib (zainstalowana w Ćwiczeniu 1) zawiera procesory dla obu celów — jeden wykrywa metadane infrastruktury automatycznie, drugi pozwala dodawać dowolne pary klucz-wartość
- Procesory muszą pojawić się w **dwóch miejscach** w konfiguracji Collectora: w bloku definicji `processors:` oraz na liście `processors:` pipeline'a — brak któregośkolwiek oznacza, że procesor nie uruchomi się
- Kolejność procesorów w pipeline ma znaczenie: `memory_limiter` musi być zawsze pierwszy, `batch` zawsze ostatni
- Aby zastosować zmiany, edytuj plik `otel-collector.yaml` i zastosuj go ponownie — operator wykryje zmianę i automatycznie wdroży nowy pod Collectora
- Poczekaj na uruchomienie nowego poda przed testowaniem

Wymagany rezultat: uruchom poniższe zapytanie TraceQL i potwierdź, że atrybut jest obecny we wszystkich spanach:


```
{span.student.group="group-KowalskiNowak"}
```

Otwórz dowolny pasujący span i sprawdź panel **Resource attributes** — powinieneś zobaczyć atrybuty `cloud.provider`, `cloud.region`, `cloud.availability_zone` i `host.name` dodane przez procesor `resourcedetection`. Atrybuty `student.group` i `workshop` pojawiają się w sekcji **Span attributes**.

Podsumowanie

W trakcie tych zajęć:

- Wdrożyłeś OTel Operator i Collector z pipeline'em wielosygnałowym (ślady, logi, metryki aplikacji) i zrozumiałeś, jak admission webhooks umożliwiają instrumentację bez modyfikacji kodu
- Włączyłeś auto-instrumentację na czterech usługach w dwóch językach i zaobserwowałeś, jak mechanizm wstrzykiwania różni się dla każdego języka na poziomie środowiska uruchomieniowego
- Wykonałeś zapytania o ślady rozproszone z TraceQL — filtrując według usługi, czasu trwania, kodu statusu i rodzaju spana w wielu zinstrumentowanych usługach
- Uruchomiłeś service graph i zbudowałeś dashboard RED (Rate, Error, Duration) ze span metrics Tempo w Prometheusie
- Skonfigurowałeś Grafanę do łączenia śladów bezpośrednio z logami i metrykami poda, który je wyprodukował
- Skonfigurowałeś kompletny łańcuch alertowania: opóźnienie spana → PrometheusRule → Alertmanager

Kompletny stos obserwowalności:

